DISEÑO DE SOFTWARE ROBUSTO

Arquitectura Hexagonal con Patrones Strategy y State



Por: Manuel Esteban Vigoya Gonzalez y
Brayan Stiven Devia Muñoz

¿Qué es la Arquitectura Hexagonal?

También conocida como Puertos y Adaptadores, aísla el núcleo de negocio de tecnologías externas.

Independencia Tecnológica

El núcleo no sabe si usa MySQL, MongoDB o APIs web. Solo ejecuta reglas puras.

Testeo Instantáneo

Pruebas unitarias ultrarrápidas sin levantar bases de datos ni servidores externos.

Crecimiento Sostenible

Cambiar frameworks o pasarelas de pago no rompe la lógica de la aplicación.

Componentes Clave: Puertos y Adaptadores

Cómo se comunica el interior con el exterior manteniendo los límites estrictos.

INTERIOR

El Núcleo (Domain)

Contiene las reglas de negocio puras, entidades y casos de uso de la aplicación.

CONTRATO

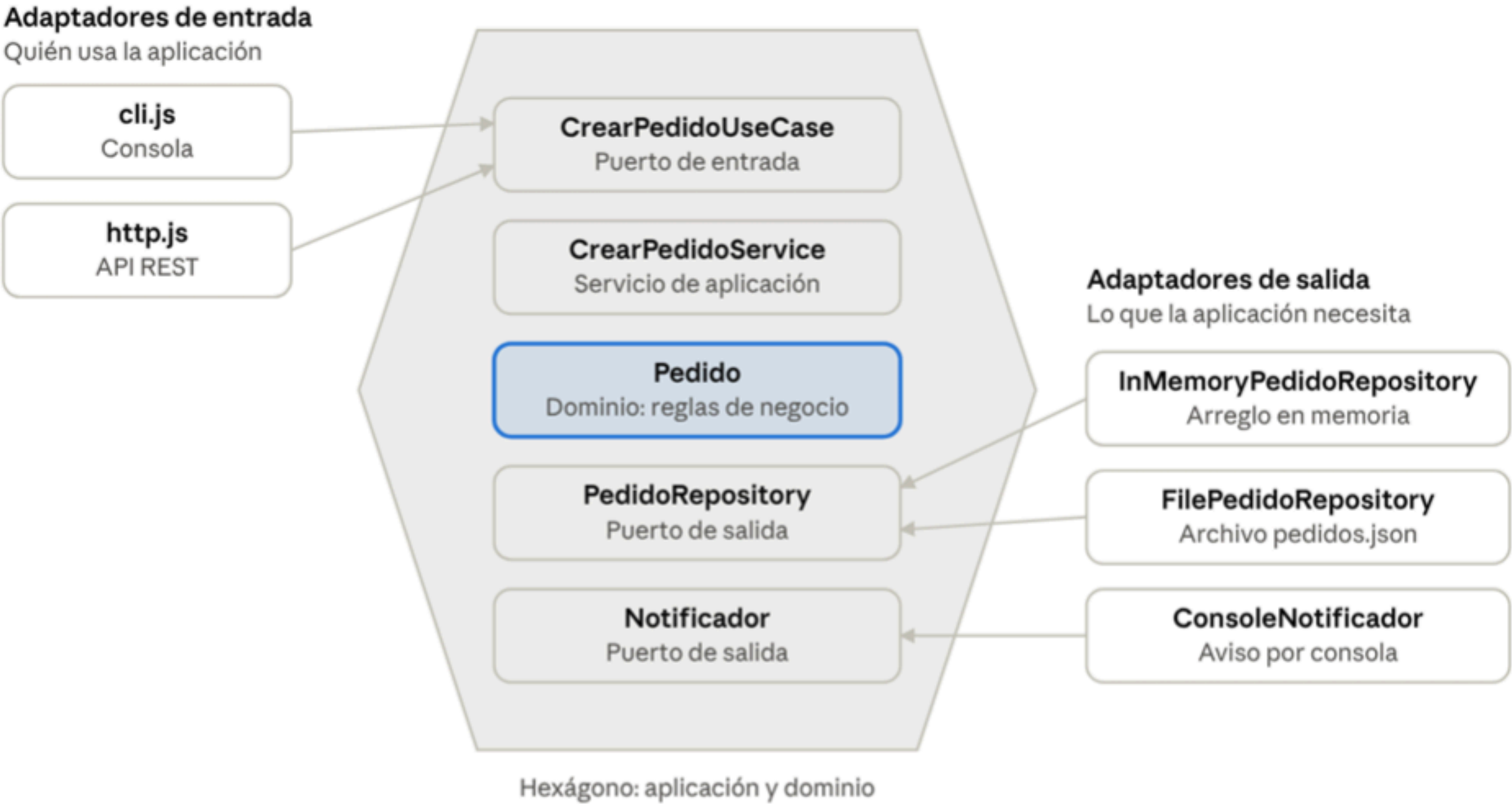
Los Puertos (Ports)

Interfaces que definen qué operaciones necesita el núcleo o qué servicios expone al exterior.

EXTERIOR

Los Adaptadores (Adapters)

Traducen tecnologías específicas (HTTP, bases de datos, UI) al lenguaje de los puertos.


Adaptadores de entrada
Quién usa la aplicación
cli.js
Consola
http.js
API REST
CrearPedidoUseCase
Puerto de entrada
CrearPedidoService
Servicio de aplicación
Pedido
Dominio: reglas de negocio
PedidoRepository
Puerto de salida
Notificador
Puerto de salida
Adaptadores de salida
Lo que la aplicación necesita
InMemoryPedidoRepository
Arreglo en memoria
FilePedidoRepository
Archivo pedidos.json
ConsoleNotificador
Aviso por consola
Hexágono: aplicación y dominio

Ventajas Y Desventajas

- **Desacoplamiento total:** El núcleo de negocio no depende de frameworks, librerías ni de la base de datos.
- **Facilidad de pruebas (Testing):** Permite probar la lógica de negocio de forma aislada mediante dobles de prueba (mocks o stubs) sin levantar infraestructura real.
- **Flexibilidad e intercambiabilidad:** Puedes cambiar un proveedor externo, una base de datos o un protocolo (por ejemplo, pasar de una API REST a consumir mensajes de Kafka) solo modificando el adaptador correspondiente.
- **Menor riesgo de regresiones:** Los cambios tecnológicos en capas externas rara vez afectan las reglas de negocio.
- **Trabajo en equipo en paralelo:** Una vez definidos los puertos (interfaces), varios equipos o desarrolladores pueden construir el núcleo y los adaptadores de manera simultánea.

- **Complejidad inicial elevada:** Requiere escribir más código de configuración, interfaces, puertos y adaptadores desde el primer día.
- **Sobreingeniería (Over-engineering):** En aplicaciones pequeñas, CRUDs sencillos o proyectos con reglas de negocio simples, crea una cantidad de capas innecesaria que dificulta el desarrollo.
- **Curva de aprendizaje:** Exige que el equipo domine conceptos como inversión de dependencias, puertos y mapeo de modelos entre capas.
- **Duplicidad de código y mapeos:** A menudo es necesario duplicar objetos (por ejemplo, entidades de dominio vs. entidades ORM/DTO) y escribir código de mapeo bidireccional entre ellos.

Si el dominio importa más que el framework, el hexágono encaja.

¿Por qué usar Patrones de Diseño en el Hexágono?

El núcleo hexagonal protege reglas, pero cuando la lógica interna crece, los patrones organizan las decisiones.

Patrón Strategy

Permite intercambiar algoritmos o reglas de cálculo (como impuestos o envíos) en tiempo de ejecución sin modificar el núcleo.

Patrón State

Maneja entidades con ciclos de vida complejos (como un pedido o factura) alterando su comportamiento según su estado actual.

Patrón Strategy: Flexibilidad de Algoritmos

Define una familia de algoritmos, encapsula cada uno y los hace intercambiables.

El Problema Cotidiano

Tienes un código lleno de condicionales if/else dependiendo del tipo de envío (Express, Estándar, Internacional).

La Solución Strategy

Creas clases separadas para cada cálculo. El núcleo solo invoca la interfaz genérica sin importar qué regla se aplique.

Strategy Integrado en el Hexágono

El cálculo de tarifas de envío viviendo puro dentro del núcleo de negocio.

1. Interfaz Strategy

Contrato puro en el core: **calcular(pedido)**.

2. Estrategias Concretas

EnvioExpress y **EnvioNacional** con cálculos independientes.

3. Inyección Limpia

El caso de uso recibe la estrategia óptima según la selección del usuario.

Patrón State: Control de Ciclos de Vida

Permite que un objeto altere su comportamiento cuando su estado interno cambia.

El Caos del Estado

Un pedido en estado Entregado no puede cancelarse. Validar esto con múltiples if vuelve el código inmanejable.

La Solución State

Cada estado es una clase que encapsula qué acciones son permitidas y hacia dónde puede transicionar.

State Protegiendo el Dominio

Reglas de negocio estrictas dentro del corazón de la arquitectura hexagonal.

Pendiente

Permite pago y cancelación. Bloquea envíos directos.

Pagado

Permite despacho. Bloquea cancelaciones directas.

Entregado

Estado final inalterable. Cero modificaciones permitidas.

Estructura de Código Limpia

Cómo conviven el núcleo, Strategy y State en un proyecto real.

Primera terminal
node --version

- npm test
- node src/main.js Ana "Camisa:50000:2" "Gorra:20000:1"
- $env:REPO="archivo"; node src/main.js Ana "Camisa:50000:2" "Gorra:20000:1"
- Get-Content pedidos.json
- node src/main.js Ana "Camisa:50000:0"
- $env:ENTRADA="http"; $env:REPO="archivo"; node src/main.js

Invoke-RestMethod -Method Post -Uri http://localhost:3000/pedidos -ContentType "application/json" -Body '{"cliente":"Luis","items":[{"nombre":"Zapatos","precio":120000,"cantidad":1}]}'
Invoke-RestMethod -Method Post -Uri http://localhost:3000/pedidos -ContentType "application/json" -Body '{"cliente":"Luis","items":[]}'

Arquitectura y Patrones en Armonía

La arquitectura hexagonal aísla el exterior, mientras que Strategy y State ordenan el interior. El resultado es un software fácil de entender, probar y escalar.

Conclusión

Aunque requiere una mayor estructura inicial y líneas de código adicionales (por el uso de interfaces y mapeadores), su verdadero valor se refleja en la sostenibilidad del proyecto en el tiempo, convirtiéndola en una inversión clave para sistemas empresariales complejos y escalables.